

0.60. Aller et revenir : aller_retour

`aller_retour(x, y, tuile, pasX, pasY, vitesse, fois)` : aller jusqu'à un point, revenir, et recommencer.

1. Ce qu'il faut comprendre

Un va-et-vient : la lettre va jusqu'à un point, revient à sa place, et recommence.

Ce qui est nouveau ici : `aller_retour(x, y, tuile, pasX, pasY, vitesse, fois)`.

- `x`, `y` : la place de départ, où la lettre revient à chaque fois.
- `pasX`, `pasY` : où est l'autre bout, comme pour `deplace` : `5, 0` veut dire 5 cases à droite, rien sur Y.
- `vitesse` : en millisecondes par pas, comme pour `carre`. `fois` : combien d'allers-retours.

Au bord de l'écran, l'autre bout est ramené dans l'écran.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <aller_retour>` → [0.59.3. La fonction aller_retour\(\) — aller et revenir](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1308 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
aller_retour() 400 o ← main() - une fonction fournie par le langage
main()         37 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases - attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() - efface des cases
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()        x 3   39 o sur place, dans aller_retour() + AdresseCase (10 o)
effacer()      x 2   30 o sur place, dans aller_retour() + AdresseCase (10 o), EffacerCases (12 o)
image()        x 3    9 o sur place, dans aller_retour(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
aller_retour() x 1   30 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <aller_retour>    // une tuile qui va et revient
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // La ligne, morceau par morceau :
    //
    // aller_retour(10, 8, ALPHABET[0], 5, 0, 250, 2);
    //
    //      | | |      | | | |
    //      | | |      | | |   +-- fois    : 2 allers-retours
    //      | | |      | |   +----- vitesse : 250 ms par pas
    //      | | |      |   +----- pasY    : 0, ne bouge pas sur Y
    //      | | |      +----- pasX    : 5, l'autre bout est
    //      | | |                      5 cases à droite
    //      | | +----- tuile    : la lettre A
    //      +---+----- x, y    : le départ (10, 8)
    //
    //      colonne : 10 11 12 13 14 15
    //      ligne 8 :  A → → → → →      l'aller : de (10, 8) à (15, 8)
    //                  ← ← ← ← ←      le retour : de (15, 8) à (10, 8)
    aller_retour(10, 8, ALPHABET[0], 5, 0, 250, 2);

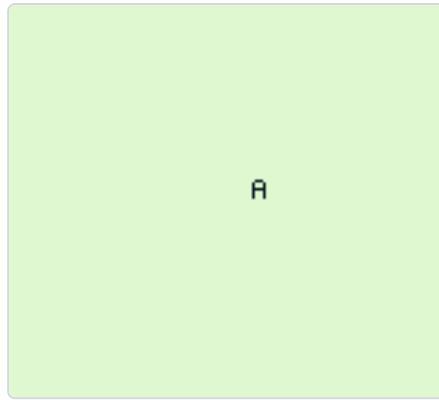
    while (true) {
        image();          // le A est revenu à (10, 8)
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



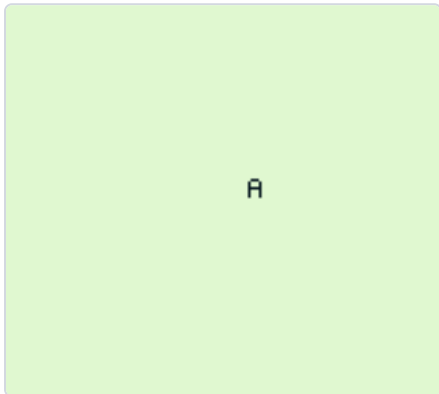
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui fait deux allers-retours de 5 cases vers la droite, et s'arrête à sa place.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ deux fois au bout de droite (15, 8) (2)

✓ jamais plus loin

✓ le A revient à sa place (10, 8) (10,8)

Ensuite : **0.60.1. Aller et revenir : aller_retour — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++